SouqAlmazad -سوق المزاد

Final Project Report

Database - Course Project

An-Najah National University

Faculty of Engineering Computer Engineering Department

Student Names	Lara Ali Saleh & Shahd Imad Daas
Submitted to	Dr. Firas Shakaa
Academic Year	2025-2026
Submission Date	May 2026

Abstract

SouqAlMazad is a full-stack multi-vendor e-commerce marketplace with an integrated real-time auction system. Built with Python 3, CustomTkinter, and MySQL 8.0, the platform enables sellers to open stores and list products for direct sale or time-bounded auction, while buyers browse, purchase, and compete through live bidding. This report documents the complete database design — including an 18-table normalized schema, ISA specialization hierarchy, indexing strategy, and referential integrity constraints — together with the application architecture, SQL query analysis, business rule enforcement, and concurrency control mechanisms. The system satisfies Third Normal Form throughout, with three deliberate and documented denormalizations for performance. Atomic transactions with row-level locking (SELECT...FOR UPDATE) protect the bid-placement and purchase flows from race conditions.

Keywords — relational database, e-commerce, auction system, MySQL, normalization, ISA hierarchy, concurrency control, Python

1. INTRODUCTION

SouqAlMazad ("The Auction Market" in Arabic) is a full-stack multi-vendor e-commerce marketplace that combines traditional fixed-price retail with a live auction module. Multiple independent sellers open stores, list products for direct sale or time-bounded auction, and buyers can purchase items outright or compete through real-time bidding.

The platform is backed by a relational MySQL 8.0 database enforcing strong referential integrity, and a Python/CustomTkinter front-end that communicates with the database exclusively through a dedicated data-access module (db_helper.py). The system was designed to demonstrate core database concepts — normalization, referential integrity, transactions, concurrency control, and role-based access — in a single realistic domain.

1.1 System Overview

Metric	Value
Database tables	18 (fully normalized, 3NF)
Custom indexes	8
SQL queries	20 (6 JOIN · 8 GROUP BY · 6 Subquery)
Python LOC — db_helper.py	~2,005
Python LOC — marketplace.py	~2,681
User roles	3 (Buyer · Seller · Admin)

Technology stack	MySQL 8.0 · Python 3 · CustomTkinter

1.2 Application Interface

Fig. 1 shows the login and home screen of SouqAlMazad as rendered by the CustomTkinter GUI layer. The interface presents a welcome panel with the project logo and live auction status, alongside a login form for returning users.

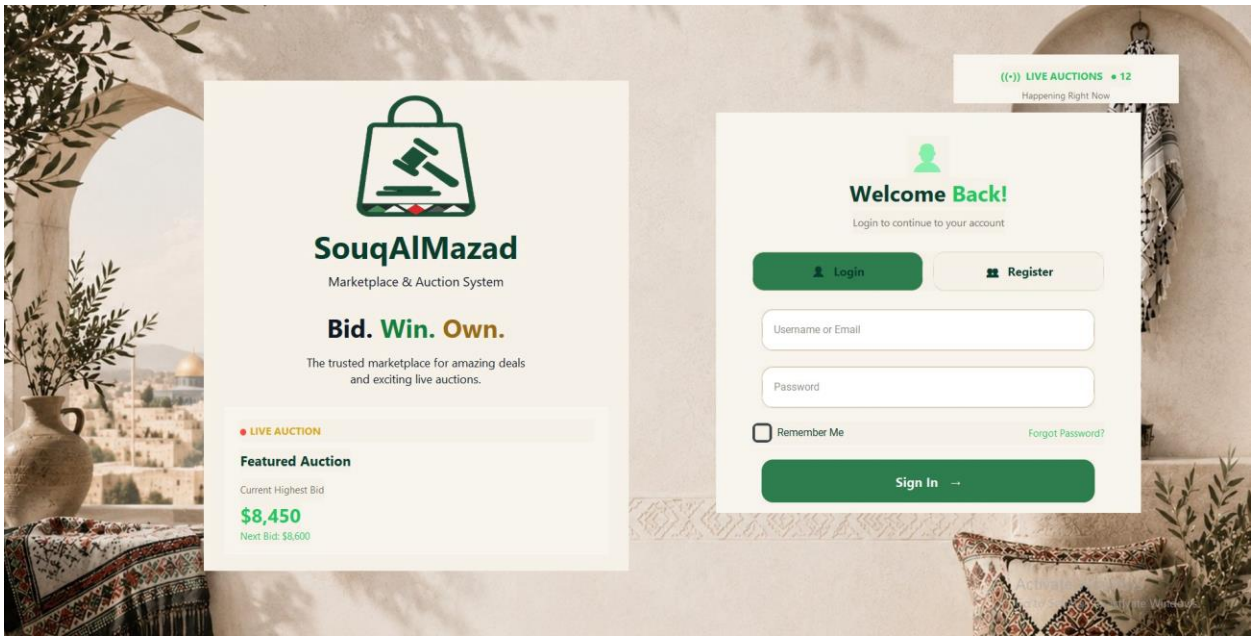

Fig. 1 — SouqAlMazad application interface

1.3 Key Contributions

- 18-table normalized schema satisfying 1NF, 2NF, and 3NF, with documented denormalizations.
- ISA specialization hierarchy: user is the supertype; buyer, seller, and admin are subtypes enforced by UNIQUE foreign keys.
- Atomic auction bidding using SELECT...FOR UPDATE to prevent race conditions under concurrent access.
- Automated auction lifecycle: close_ended_auctions transitions auctions and creates winner orders transactionally.
- Soft-delete pattern (is_active flag) preserving historical orders, reviews, and bids.
- Strict separation of concerns: all SQL in db_helper.py; GUI in marketplace.py constructs no SQL strings.

2. DATABASE DESIGN

2.1 Entity-Relationship Model

Fig. 2 presents the Entity-Relationship Diagram (ERD) for SouqAlMazad. The schema organizes 18 entities into five logical clusters: User Identity, Catalogue, Auction, Order Lifecycle, and Social/Moderation. Solid lines denote one-to-many (1:N) relationships, red dashed lines denote many-to-many (M:N) relationships, and blue dashed lines indicate the ISA generalization hierarchy.

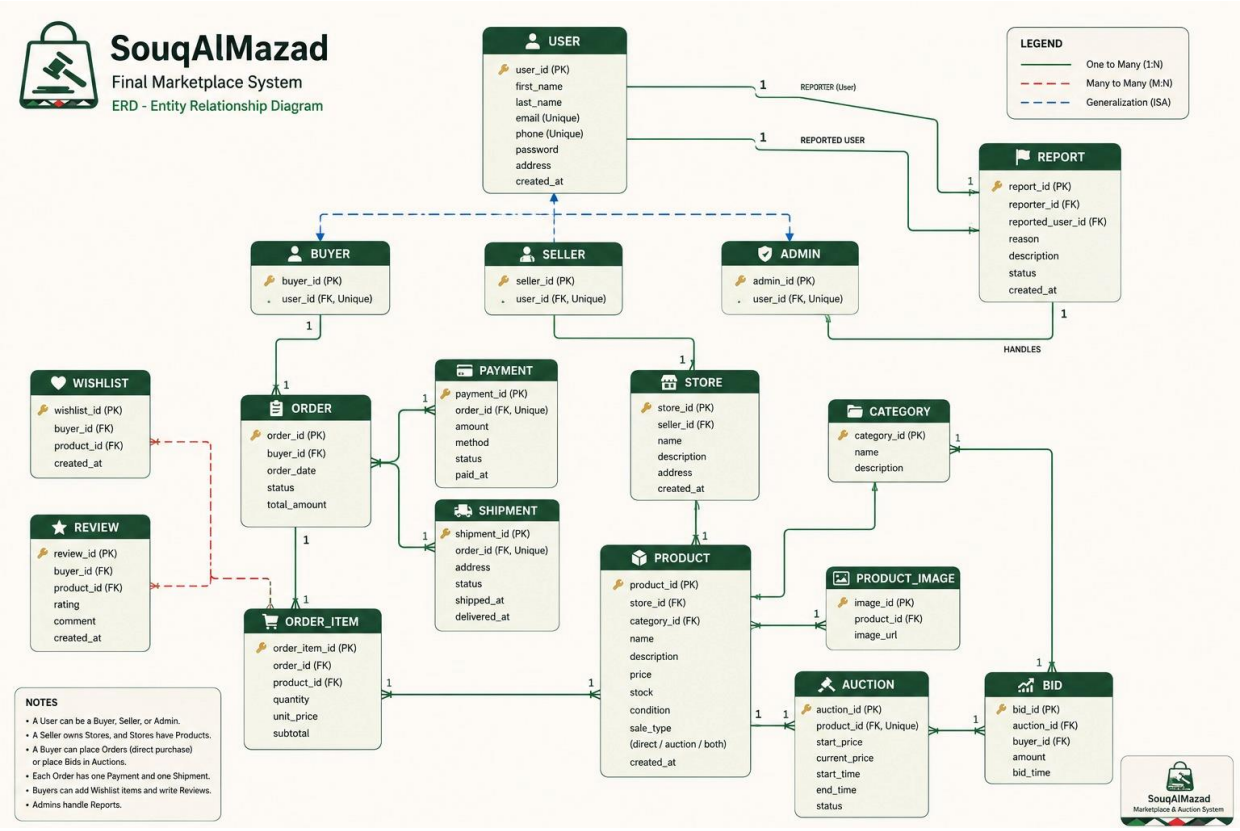

Fig. 2 — Entity-Relationship Diagram. Entities are grouped by functional cluster

2.2 Entity Clusters

Cluster	Tables	Key Relationship
User Identity	user, buyer, seller, admin	ISA hierarchy — UNIQUE FK from each subtype to user
Catalogue	store, category, product, product_image	seller → store → product → product_image
Auction	auction, bid	product → auction ← bid

Order Lifecycle	order, order_item, payment, shipment	4-table composite — all CASCADE on order_id
Social / Moderation	review, wishlist, notification, report	review uses 3-FK verified-purchase design

2.3 ISA Specialization Hierarchy

The user table is the supertype of the ISA hierarchy. Each subtype (buyer, seller, admin) carries a UNIQUE foreign key on user_id, ensuring at most one subtype row per user. ON DELETE CASCADE acts as a safety net; the application uses soft-delete (is_active = FALSE) as the primary mechanism to preserve historical data.

```
CREATE TABLE seller (
seller_id        INT PRIMARY KEY AUTO_INCREMENT,
user_id          INT NOT NULL UNIQUE,   -- ISA link (enforces 1:1)
business_name    VARCHAR(100),
commission_rate  DECIMAL(4,2) DEFAULT 5.00,
is_verified      BOOLEAN DEFAULT FALSE,
FOREIGN KEY (user_id) REFERENCES `user`(user_id) ON DELETE CASCADE
);
```

The do_login function verifies the subtype row exists after authenticating the user — rejecting the session with an ISA integrity error if the link is broken. _ensure_subtype_for_user enforces this on every write.

2.4 Normalization Analysis

Normal Form	Proof
1NF	All columns atomic. Repeating groups eliminated — product_image is a separate table, not an array inside product.
2NF	All tables use single-column surrogate PKs, eliminating any possibility of partial dependencies.
3NF	No transitive dependencies — store_name lives in store, not in product; category name lives in category, not duplicated.

Documented Denormalizations

- auction.current_highest — cached aggregate allowing place_bid to compare with a single FOR UPDATE read instead of a MAX() scan. Updated atomically on every accepted bid.
- auction.winner_buyer_id — cached to avoid re-scanning bids at auction close. Uses ON DELETE SET NULL to preserve auction records if a buyer account is removed.
- store.rating_avg — cached aggregate updated by add_review, saving a JOIN + GROUP BY on every store listing request.

2.5 Key Design Decisions

Table / Column	Decision	Rationale
product.sale_type	ENUM('direct','auction','both')	Drives dispatch logic; create_auction validates sale_type before INSERT.
auction.winner_buyer_id	ON DELETE SET NULL	Preserve auction history if buyer account is deleted — asymmetric cascade intentional.
order (triple amounts)	total + discount + final	Audit requirement: all three must remain independently readable.
review (3-FK design)	product_id + buyer_id + order_id	Forces verified-purchase check inside add_review before INSERT.
report.reported_type	Polymorphic ENUM + reported_id	Single report table for users/products/stores; FK enforcement pushed to add_report.

3. DATABASE IMPLEMENTATION

3.1 Schema Summary

Table II documents every table with its primary key, key columns, and the most important constraint or design note. Full DDL is provided in the file DATAScript.sql.

Table	PK / Key Columns	Constraints & Notes
user	user_id	UNIQUE(email) · is_active soft-delete · user_type ENUM
buyer	buyer_id, user_id FK UNIQUE	CASCADE on user · loyalty_points · gender ENUM
seller	seller_id, user_id FK UNIQUE	CASCADE on user · commission_rate DEFAULT 5.00 · is_verified
admin	admin_id, user_id FK UNIQUE	CASCADE on user · role ENUM('super_admin','moderator','support')
category	category_id	UNIQUE(name) · delete_category validates no active products
store	store_id, seller_id FK	CASCADE on seller · rating_avg cached aggregate
product	product_id, store_id FK, category_id FK	sale_type ENUM · stock_quantity · is_active · view_count
product_image	image_id, product_id FK	CASCADE on product · is_primary boolean · display_order

auction	auction_id, product_id FK	current_highest cached · winner_buyer_id SET NULL · status ENUM
bid	bid_id, auction_id FK, buyer_id FK	CASCADE on auction · is_winning managed by place_bid
order	order_id, buyer_id FK	7-status lifecycle · total/discount/final amounts
order_item	order_item_id, order_id FK	CASCADE on order · store_id FK for revenue attribution
payment	payment_id, order_id FK	CASCADE on order · payment_method & status ENUMs
shipment	shipment_id, order_id FK	CASCADE on order · carrier + tracking + status ENUM
review	review_id; product_id + buyer_id + order_id	CASCADE on product · CHECK(rating 1–5) · verified-purchase in app
wishlist	wishlist_id, buyer_id + product_id	UNIQUE(buyer_id, product_id) prevents duplicates
notification	notification_id, user_id FK	CASCADE on user · type ENUM 6 values · is_read flag
report	report_id, reporter_id FK	reported_type ENUM + reported_id polymorphic design

3.2 Indexing Strategy

MySQL automatically indexes PRIMARY KEY and UNIQUE columns. Table III lists the eight additional indexes created on high-selectivity columns that appear in WHERE/JOIN clauses of frequently-called queries.

Index Name	Column(s)	Queries Accelerated
idx_product_store	product(store_id)	Store product listings, Q5
idx_product_category	product(category_id)	Category browsing, Q15 correlated subquery
idx_auction_status	auction(status)	Active auction filter, Q11
idx_bid_auction	bid(auction_id)	Bid history lookups, Q14
idx_order_buyer	order(buyer_id)	Buyer order history, Q10
idx_order_status	order(status)	Dashboard analytics, Q13

idx_notification_user	notification(user_id)	User notification feed
idx_review_product	review(product_id)	Review display, Q8 average rating

4. SQL QUERY ANALYSIS

Twenty queries were developed to exercise the schema across three categories. Table IV provides a reference for all queries; the subsections below present selected queries with design commentary.

#	Query Description	Type	Key Technique
Q1	Product catalog with store and category	JOIN	INNER JOIN — both FKs are NOT NULL
Q2	Bid history with buyer names	JOIN	5-way ISA traversal: bid→buyer→user
Q3	Order master view	JOIN	INNER on items; LEFT on payment/shipment
Q4	Reviews with reviewer and product	JOIN	ISA traversal: review→buyer→user
Q5	Sellers with store and product count	JOIN	LEFT JOIN preserves empty stores
Q6	Wishlist with buyer name and product	JOIN	Bridge table traversal
Q7	Products per store	GROUP BY	LEFT JOIN + COUNT; stores with 0 included
Q8	Average rating per product	GROUP BY	HAVING COUNT > 0 filters unreviewed
Q9	Revenue per store	GROUP BY	SUM via order_item.store_id
Q10	Buyer order count and total spend	GROUP BY	ISA traversal + GROUP BY buyer
Q11	Active auctions per category	GROUP BY	WHERE status='active' before GROUP
Q12	Most wishlisted products	GROUP BY	ORDER BY + LIMIT pattern

Q13	Orders by status	GROUP BY	Dashboard aggregate; no join needed
Q14	Bid count per auction	GROUP BY	LEFT JOIN preserves 0-bid auctions
Q15	Products above category average price	Subquery	Correlated — inner refs outer category_id
Q16	Heavy bidders (above average)	Subquery	Derived table + scalar subquery
Q17	Stores that never sold anything	Subquery	NOT IN anti-join on order_item
Q18	Products with no reviews	Subquery	NOT EXISTS — safe with potential NULLs
Q19	Per-auction highest bid and bid count	Subquery	Two scalar subqueries in SELECT list
Q20	Above-average spenders	Subquery	Two-level nested aggregate

4.1 JOIN Queries — Q3: Order Master View

Q3 combines INNER and LEFT joins deliberately. INNER joins on buyer and order_item are correct because those relationships are mandatory. LEFT joins on payment and shipment preserve new orders that exist before their child rows are created — an INNER join would silently exclude them.

```
SELECT o.order_id, u.first_name AS buyer, p.title AS product,
       oi.quantity, oi.subtotal,
       pay.payment_method, pay.status AS pay_status,
       sh.carrier_name,  sh.status  AS ship_status
FROM `order` o
INNER JOIN buyer      bu  ON o.buyer_id   = bu.buyer_id
INNER JOIN `user`      u  ON bu.user_id    = u.user_id
INNER JOIN order_item oi  ON o.order_id    = oi.order_id
INNER JOIN product     p  ON oi.product_id = p.product_id
LEFT  JOIN payment   pay  ON o.order_id    = pay.order_id
LEFT  JOIN shipment   sh  ON o.order_id    = sh.order_id;
```

4.2 GROUP BY Queries — Q8 and Q9

Q8 demonstrates the HAVING clause — the only correct mechanism for filtering on an aggregate. WHERE executes before grouping, making COUNT unavailable; HAVING executes after. Q9 attributes revenue via order_item.store_id rather than order.buyer_id, because a single order can span multiple stores.

```
-- Q8: Average rating per product (at least one review)
SELECT p.title, AVG(r.rating) AS avg_rating, COUNT(r.review_id) AS reviews
FROM product p LEFT JOIN review r ON p.product_id = r.product_id
GROUP BY p.title  HAVING COUNT(r.review_id) > 0;
```

```
-- Q9: Revenue per store, excluding cancelled orders
SELECT s.store_name, SUM(oi.subtotal) AS total_sales
FROM store s
INNER JOIN order_item oi ON s.store_id  = oi.store_id
INNER JOIN `order`     o ON oi.order_id = o.order_id
WHERE o.status != 'cancelled'
GROUP BY s.store_name  ORDER BY total_sales DESC;
```

4.3 Subqueries — Q15 and Q20

Q15 uses a correlated subquery: the inner AVG re-executes per outer row because it references the outer query's p.category_id. Q20 is the most complex query in the project — a derived table computes per-buyer spend, and a second nested derived table re-aggregates those totals. The same GROUP BY executes twice; a CTE (WITH spending AS ...) would compute it once at production scale.

```
-- Q15: Products above their category average price (correlated)
SELECT p.title, p.price, c.name AS category
FROM product p INNER JOIN category c ON p.category_id = c.category_id
WHERE p.price > (
    SELECT AVG(p2.price) FROM product p2
    WHERE p2.category_id = p.category_id
);
```

```
-- Q20: Buyers who spent above the platform average (two-level aggregate)
SELECT u.first_name, u.last_name, spending.total
FROM (
    SELECT buyer_id, SUM(final_amount) AS total
    FROM `order` WHERE status != 'cancelled' GROUP BY buyer_id
) AS spending
INNER JOIN buyer bu ON spending.buyer_id = bu.buyer_id
INNER JOIN `user`  u ON bu.user_id = u.user_id
WHERE spending.total > (
    SELECT AVG(t.total) FROM (
        SELECT SUM(final_amount) AS total FROM `order`
        WHERE status != 'cancelled' GROUP BY buyer_id
    ) AS t
);
```

5. APPLICATION ARCHITECTURE

5.1 Three-Tier Structure

SouqAlMazad follows a strict three-tier architecture. The presentation tier (souqalmazad_marketplace.py, ~2,681 lines) contains all CustomTkinter GUI code and imports no database libraries. The data-access tier (db_helper.py, ~2,005 lines) contains all SQL and uses %s parameter binding universally, eliminating SQL injection risk. The persistence tier is the MySQL 8.0 database.

```
PRESENTATION   souqalmazad_marketplace.py  (~2,681 LOC)
               CustomTkinter: Buyer / Seller / Admin dashboards
               ↓ function calls only — zero SQL strings
DATA-ACCESS    db_helper.py  (~2,005 LOC)
               %s parameter binding — no string concatenation
```

```
                Transactions + SELECT...FOR UPDATE
                ↓ parameterized SQL
PERSISTENCE     MySQL 8.0 — 18 tables, 8 custom indexes
```

5.2 Concurrency Control — place_bid

Bid placement, direct purchase, and auction closure each modify shared state that concurrent users may access simultaneously. All three operations use pessimistic locking via SELECT...FOR UPDATE. If any step raises an exception, the transaction rolls back atomically, leaving no partial state.

```
# place_bid — pessimistic row-level locking
cur.execute("""
    SELECT auction_id, current_highest, status
    FROM auction WHERE auction_id = %s FOR UPDATE""",
    (auction_id,))
auc = cur.fetchone()
if auc['status'] != 'active':
    raise ValueError('Auction is not active')
if bid_amount <= float(auc['current_highest'] or 0):
    raise ValueError('Bid must exceed current highest')
# INSERT bid → UPDATE auction.current_highest → COMMIT
```

5.3 Business Logic Summary

Function	Responsibility
place_bid	Lock auction row → validate → INSERT bid → UPDATE current_highest → COMMIT
direct_buy_product	Lock product row → check stock → INSERT order + payment + shipment → COMMIT
close_ended_auctions	Scan upcoming→active→ended; create winner orders transactionally; notify participants
deactivate_seller_by_user_id	Soft-delete user→stores→products→auctions in one transaction; preserve history
_sync_order_related_statuses	Keep order / payment / shipment status consistent; auto-complete on 'delivered'
add_review	Pre-validate purchase via JOIN before INSERT; enforce CHECK(rating 1–5)
seller_add_product	Verify store ownership at data layer before INSERT — not merely in the UI

5.4 Role-Based Access Control

Role	Permitted Operations
Buyer	Browse, bid, direct purchase, wishlist, reviews, order tracking, notifications, reports
Seller	Create/manage stores and products, create auctions, fulfill orders, view analytics

Admin	Manage all users, verify sellers, resolve reports, manage categories, platform analytics

6. BUSINESS RULES AND TESTING

6.1 Database-Level Constraints

Rule	Mechanism
Email uniqueness	UNIQUE(email) on user table
One subtype per user	UNIQUE(user_id) on buyer, seller, admin
Rating range 1–5	CHECK(rating BETWEEN 1 AND 5) on review
Unique wishlist entry	UNIQUE(buyer_id, product_id) on wishlist
Referential integrity	FK constraints with explicit CASCADE / SET NULL / RESTRICT
Order child cascade	order_item, payment, shipment CASCADE on order_id
Auction history preservation	auction.winner_buyer_id ON DELETE SET NULL

6.2 Application-Level Constraints

Rule	Function	Enforcement Logic
Bid must exceed current	place_bid	Read current_highest with FOR UPDATE; compare before INSERT
One active auction per product	create_auction	COUNT active auctions for product_id before INSERT
Sale type must allow auction	create_auction	Validates sale_type IN ('auction','both')
Verified purchase for reviews	add_review	JOIN order_item→order confirms buyer purchased via that order
Category not deletable with products	delete_category	COUNT active products before DELETE
Seller ownership	seller_add_product / seller_create_auction	Verify store.seller_id = calling seller_id

6.3 Test Scenarios

- Login rejects deactivated accounts (is_active = FALSE) and sessions with broken ISA subtype links.

- Concurrent bid simulation: two transactions with BEGIN / SELECT...FOR UPDATE confirm the second blocks until the first commits.
- Auction close: running close_ended_auctions after expiring an auction creates order + payment + shipment rows for the winner.
- Ownership enforcement: calling seller_add_product with a foreign store_id raises ValueError at the data layer, independent of the GUI.
- Review integrity: submitting a review for a product not in the buyer's orders raises ValueError before any INSERT.

6.4 Future Work

- Rewrite Q20 as a CTE (WITH spending AS ...) — MySQL 8.0 supports CTEs, eliminating the duplicated GROUP BY.
- Replace the polymorphic report.reported_id with three nullable FKs and a CHECK constraint to push referential integrity into the database engine.
- Schedule close_ended_auctions on a background timer rather than triggering it from the GUI.
- Add load testing for the FOR UPDATE lock path to quantify throughput under high-concurrency bidding.

7. CONCLUSION

SouqAlMazad demonstrates that a properly normalized relational schema with deliberate, documented trade-offs can support a realistic multi-vendor marketplace without sacrificing data integrity or application performance. The 18-table design captures every meaningful entity in the domain, models the ISA specialization hierarchy correctly, supports both direct-purchase and auction sale of the same product, and preserves historical integrity through a deliberate soft-delete pattern.

The 20 SQL queries exercise the schema across JOIN traversal, GROUP BY aggregation, and nested subquery patterns, reflecting the kinds of analytical queries a production marketplace would require. On the application side, the strict separation between the CustomTkinter GUI and db_helper.py ensures that SQL is auditable in one location, parameter binding is universal, and the most concurrency-sensitive operations — bid placement, direct purchase, and auction closure — are protected by explicit transactions with row-level locking.

The role-based dashboards give Buyers, Sellers, and Administrators exactly the operations they require. Ownership rules are enforced at the data-access layer rather than relying on the GUI alone, meaning that direct API calls or GUI bypass attempts cannot circumvent the authorization logic.